Assumptions and Constraints

📌 . Assumptions

- **Order Ownership:** Orders are tied to a single customer with an immutable `customerId`.
- **Business Scope:** Prices are in Euros (€); payment processing and end-user customer status updates are explicitly out of scope (only internal staff update states).
-
- **Order Lifecycle:** All orders automatically initialize in the `PENDING` state and follow strict transition rules.
- **Technical Baseline:** Built with Python 3.12 for low-volume educational/demo purposes with no implied production SLA.

⚠️ Constraints

Project Boundaries: Strictly 4-week delivery window managed by 1–2 developers.

Resource & Cost Limits: Restricted to **5 core AWS services** (API Gateway, Lambda, DynamoDB, IAM, CloudWatch) strictly within the **AWS Free Tier**.

Zero Idle Cost Architecture: Must use serverless pay-per-request infrastructure (no continuously running application servers).

Database & Concurrency Rules:

- `GET /orders` **must** include a `customerId` filter to prevent costly database scans (`SCAN`).
- `PATCH` status updates **must** include `expectedStatus` and `expectedVersion` to enforce optimistic locking and block stale overwrites.